

Program 1: Understanding Constructor Chaining and Method Overloading

1. Which constructor is called first when new Learn() is created?

Default constructor

2. What is constructor chaining in this code? Which line shows this?

Constructor chaining is a process where one constructor calls another constructor within the same class to reuse code and avoid duplication.

Line: this("Smart");

3. Identify which part of the code shows method overloading.

Both method which are show

4. How many times is the constructor invoked when you run the program?

2 times

5. What is the output of this program?

Learn, Smart

Default Constructor

Show: Focus

Program 2 : Analysing Constructor and Method Overloading

1. How many constructors are defined in the class?
2 constructors
2. Which constructor is called first when new Calculator() is used?
Default constructor or Calculator()
3. What is the purpose of this(20, 10); in the default constructor?
To call another constructor which is Calculator(int x, int y)
4. How is constructor chaining applied in this program?
By using this() keyword inside a constructor
5. Identify the method overloading in this code.
calculate method
6. How many times are methods named calculate executed in main()?
2 times
7. What is the expected output of the program?

[Constructor] Sum: 30

[Constructor] Difference: 10

[Method] Sum of 5 and 15: 20

[Method] Sum of 1, 2, and 3: 6

Program 4 : Carpet room case study

```
package chap2;

import java.util.Scanner;

class Room {
    private double length;
    private double width;

    public Room(double length, double width) {
        this.length = length;
        this.width = width;
    }

    public double getArea() {
        return length * width;
    }

    @Override
    public String toString() {
        return "Room [Length = " + length + ", Width = " + width + ", Area = " + getArea() + " sq. meters]";
    }

    public boolean equals(Room otherRoom) {
        return this.getArea() == otherRoom.getArea();
    }
}

class Carpet {
    private double pricePerSqMeter;

    public Carpet(double price) {
        this.pricePerSqMeter = price;
    }

    public double getPrice() {
        return pricePerSqMeter;
    }

    @Override
    public String toString() {
        return "Carpet [Price per square meter = RM " + pricePerSqMeter + "]";
    }
}
```

```

class RoomCarpet {
    private Room room;
    private Carpet carpet;

    public RoomCarpet(Room room, Carpet carpet) {
        this.room = room;
        this.carpet = carpet;
    }

    public double getTotalCost() {
        return room.getArea() * carpet.getPrice();
    }

    @Override
    public String toString() {
        return room.toString() + "\n" + carpet.toString() + "\nTotal Carpet Cost = RM " +
getTotalCost();
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        // Get room dimensions
        System.out.print("Enter room length (in meters): ");
        double length = input.nextDouble();

        System.out.print("Enter room width (in meters): ");
        double width = input.nextDouble();

        // Get carpet price
        System.out.print("Enter carpet price per square meter: RM ");
        double price = input.nextDouble();

        // Create objects
        Room room = new Room(length, width);
        Carpet carpet = new Carpet(price);
        RoomCarpet roomCarpet = new RoomCarpet(room, carpet);

        // Display details
        System.out.println("\n=== Room Carpet Summary ===");
        System.out.println(roomCarpet);

        // Testing equals method
        System.out.println("\nIs this room equal to another room with the same size?");
    }
}

```

```
Room anotherRoom = new Room(length, width);  
System.out.println(room.equals(anotherRoom)); // should return true
```

```
    input.close();  
}  
}
```